

Licence MIASHS première année

Rapport de projet informatique

Moteur de Simulation Financière et Analyse de Risque

Projet réalisé en Mai 2026

Étudiant :

Lucas Skhiri (N° Étudiant : 45002949)

Dépôt GitHub Public :

<https://github.com/lucasskhiri/Simulateur-Risque-Financier>

Remerciements

Je tiens à remercier l'équipe pédagogique de la licence MIASHS pour l'enseignement de cette première année, qui m'a permis d'acquérir les bases fondamentales de la programmation en langage C.

Table des matières

1	Introduction	4
2	Environnement de travail	4
3	Description du projet et objectifs	4
3.1	Objectif économique	4
3.2	Objectifs techniques informatiques	4
4	Bibliothèques, Outils et technologies	4
5	Travail réalisé	5
5.1	Fonctionnalités prévues et état d'avancement	5
5.2	Répartition du travail	5
6	Difficultés rencontrées	5
7	Bilan	5
7.1	Conclusion	5
7.2	Perspectives	5
8	Bibliographie	7
9	Webographie	8
10	Annexes	9
A	Cahier des charges	9
B	Exemple d'exécution du projet	9
C	Manuel utilisateur	9

1 Introduction

Dans le cadre de la validation de la première année de licence MIAHS (Mathématiques et Informatique Appliquées aux Sciences Humaines et Sociales) et Économie-Gestion, il nous a été demandé de réaliser un projet informatique complet en langage C.

J'ai choisi de développer un **Moteur de Simulation Financière et d'Analyse de Risque**. Ce choix s'inscrit parfaitement dans ma double licence, car il allie la complexité algorithmique (gestion de la mémoire, pointeurs) à une problématique économique concrète : l'évaluation du coût d'un prêt bancaire soumis à des taux d'intérêts variables, simulée grâce à l'algorithme mathématique de Monte-Carlo.

2 Environnement de travail

Pour mener à bien ce projet, j'ai mis en place un environnement de développement structuré :

- **Éditeur et Compilateur** : Utilisation de l'IDE *Code : :Blocks* avec le compilateur GCC pour écrire, tester et déboguer le code C.
- **Gestion de versions** : Utilisation de *GitHub* pour héberger le code source, garantir la sauvegarde du projet et permettre au jury d'y accéder publiquement.
- **Rédaction** : Utilisation de la plateforme *Overleaf* pour rédiger ce rapport en L^AT_EX.

3 Description du projet et objectifs

3.1 Objectif économique

L'objectif principal est de fournir un outil capable de calculer les mensualités d'un prêt (amortissement) et d'estimer les risques liés à la fluctuation des taux d'intérêt sur plusieurs années pour un portefeuille contenant plusieurs emprunts.

3.2 Objectifs techniques informatiques

Sur le plan technique, ce projet vise à démontrer l'acquisition des compétences de la L1 :

- Création d'une architecture modulaire (fichiers `.c` et `.h`).
- Manipulation de structures (`struct`) complexes.
- Utilisation de l'allocation dynamique de la mémoire (`malloc`, `free`) via des listes chaînées pour stocker un nombre infini de prêts.

4 Bibliothèques, Outils et technologies

Le programme s'appuie sur les bibliothèques standard du C : `<stdio.h>`, `<stdlib.h>` (pour la mémoire), et `<math.h>` (pour les calculs de puissance des taux d'intérêt et l'algorithme de Box-Muller).

Utilisation de l'Intelligence Artificielle : Conformément aux directives, une IA a été sollicitée pour assister le développement. Elle a été utilisée comme un tuteur pour :

- Comprendre et structurer la logique des listes chaînées (pointeurs vers la structure suivante).
- Rédiger la formule mathématique complexe de la variation aléatoire des taux (algorithme de Monte-Carlo).
- Identifier la source de certaines erreurs de compilation sous Code : :Blocks.

5 Travail réalisé

5.1 Fonctionnalités prévues et état d'avancement

- **Menu interactif dans la console** : Réalisé.
- **Ajout dynamique de prêts (Listes chaînées)** : Réalisé. Le programme alloue de la mémoire pour chaque nouveau prêt.
- **Simulation Monte-Carlo sur 10 000 itérations** : Réalisé. Le programme calcule le pire scénario, le meilleur scénario et la moyenne espérée.
- **Interface Graphique (Fenêtre)** : Non réalisé. Le projet a été maintenu en mode Console (Terminal) car l'apprentissage de bibliothèques graphiques externes (comme SDL ou GTK) dépassait le cadre du programme de L1 et les délais impartis.

5.2 Répartition du travail

Étant seul sur ce projet, j'ai assumé **100% de la charge de travail**, de la conception de l'algorithme à la rédaction du rapport $\text{\LaTeX}$, en passant par la configuration du dépôt GitHub.

6 Difficultés rencontrées

Les principales difficultés ont été d'ordre technique concernant la gestion de la mémoire. La manipulation des pointeurs dans les listes chaînées a provoqué plusieurs "Erreurs de Segmentation" (Segmentation Fault) au début du développement, notamment lors de la libération de la mémoire (`free`) en fin de programme. Une autre difficulté a été la prise en main du langage $\text{\LaTeX}$ sur Overleaf pour formater correctement ce rapport et résoudre les erreurs de compilation liées au fichier de la page de garde.

7 Bilan

7.1 Conclusion

Ce projet marque l'aboutissement de ma première année en MIA SHS. Il m'a permis de consolider mes acquis en programmation procédurale et de comprendre comment l'informatique peut se mettre au service des mathématiques financières. L'assistance par IA a été un atout précieux pour surmonter les blocages techniques complexes.

7.2 Perspectives

Pour améliorer ce logiciel à l'avenir, il serait intéressant d'ajouter une fonction d'import/export de fichiers `.csv`. Cela permettrait de lire un grand volume de données

bancaires depuis Excel directement dans la liste chaînée du programme C, et d'exporter les résultats des simulations pour en faire des graphiques.

8 Bibliographie

- [KR88] B. KERNIGHAN, D. RITCHIE, *Le langage C*, Masson, 2ème édition, 1988
- [LAM94] L. LAMPORT, *TEX : A Document preparation system*, Addison-Wesley, 1994

9 Webographie

[GITHUB] <https://github.com/lucasskhiri/Simulateur-Risque-Financier>

[OPENCLASSROOMS] Apprenez à programmer en C, <https://openclassrooms.com/fr/courses/19980-apprenez-a-programmer-en-c>

10 Annexes

Annexe A : Cahier des charges

Le programme doit tourner dans une console. Il doit proposer un menu permettant la saisie d'un capital, d'un taux et d'une durée, et pouvoir simuler des scénarios de risques via des mathématiques aléatoires en stockant les données en mémoire dynamique.

Annexe B : Exemple d'exécution du projet

Voici les captures d'écran démontrant le bon fonctionnement du programme dans la console (compilé via Code : :Blocks) :

```
--- MENU PRINCIPAL ---
1. Ajouter un pret au portefeuille
2. Afficher le portefeuille
3. Lancer la simulation Monte-Carlo sur le dernier pret
4. Quitter
Votre choix : 1

Capital emprunte (Euros) : 200000
Taux de base annuel (%) : 5
Duree totale (en mois) : 48
Taux variable ? (1 pour Oui, 0 pour Non) : 1
[+] Pret ajoute avec succes.
```

FIGURE 1 – Exécution du menu principal et ajout d'un prêt

```
--- MENU PRINCIPAL ---
1. Ajouter un pret au portefeuille
2. Afficher le portefeuille
3. Lancer la simulation Monte-Carlo sur le dernier pret
4. Quitter
Votre choix : 3
Lancement de 10 000 simulations...

--- RESULTATS MONTE-CARLO PRENT ID 1 (10000 iterations) ---
Cout total moyen espere : 221078.73 Euros
Meilleur scenario (baisse): 217937.98 Euros
Pire scenario (hausse) : 223868.77 Euros
```

FIGURE 2 – Résultats de l'algorithme de Monte-Carlo sur 10 000 itérations

Annexe C : Manuel utilisateur

Pour exécuter le programme :

1. Ouvrir le projet dans Code : :Blocks ou compiler manuellement avec la commande `make` ou `gcc main.c finance.c -o simulateur -lm`.

2. Lancer l'exécutable.
3. Taper **1** et appuyer sur Entrée pour enregistrer un nouveau prêt.
4. Suivre les instructions à l'écran (entrer le capital, le taux et la durée).
5. Taper **3** pour lancer la simulation de risques sur ce prêt et lire les résultats.
6. Taper **4** pour quitter proprement le programme et libérer la mémoire.